Cheat sheet Kafka untuk Node.js/NestJS

Kafka 4.4 · `@confluentinc/kafka-javascript` 1.11 (API kompatibel KafkaJS) · NestJS 12 (`kafkajs` 2.2.4) · nilai di bawah sama dengan yang dipakai di lab.

Topic

Setting	Nilai	Catatan
partisi	3 (atau sesuai throughput terukur)	Batas consumer aktif per group
replication factor	3	Tahan 1 broker mati
`min.insync.replicas`	2	Produce ditolak broker kalau ISR < 2
`cleanup.policy`	`delete` (event), `compact` (state terkini per key)	
`retention.ms`	default 7 hari	Batas replay
`auto.create.topics.enable`	`false` di broker	Buat topic eksplisit

Penamaan: `<domain>.<event>.v<versi>`, retry `<topic>.<group>.retry`, DLT `<topic>.<group>.dlt`. Key: `orderId` agar urutan per order terjaga.

Producer (klien Confluent)

```
const producer = kafka.producer({
  kafkaJS: { acks: -1, idempotent: true, maxInFlightRequests: 5
        },
});
await producer.send({ topic, messages: [{ key: orderId, value:
        JSON.stringify(envelope), headers }] });
```

- `acks: -1` = `acks=all`: tunggu semua replica di ISR.
- `idempotent` default **false** di klien Confluent; nyalakan manual.
- Transaksi: `kafkaJS: { transactionalId: '<unik per instance>' }` lalu `producer.transaction()`, `tx.send`, `tx.sendOffsets({ consumer, topics })`, `tx.commit()`.
- Kirim beberapa pesan lalu tunggu, jangan `await` satu per satu kalau butuh throughput.

Consumer (klien Confluent)

```
const consumer = kafka.consumer({
  kafkaJS: { groupId: 'payment-service', fromBeginning: true,
        autoCommit: false },
  'group.protocol': 'classic', // atau 'consumer' (KIP-848)
});
await consumer.subscribe({ topics: ['order.created.v1'] });
await consumer.run({ eachMessage: async ({ topic, partition,
        message }) => {
  await prosesDanSimpanKeDb(message);           // side effect
        dulu
  await consumer.commitOffsets([{ topic, partition,
    offset: (BigInt(message.offset) + 1n).toString() }]);
}});
```

- Klien Confluent: `fromBeginning`, `autoCommit`, `autoCommitInterval` ada di **config consumer**, bukan di `subscribe()` atau `run()`. Default `autoCommit: true`, interval 5 detik.
- `group.protocol` ditulis **di luar** blok `kafkaJS`.
- Baca hasil transaksi: `kafkaJS: { readUncommitted: false }` (= `read_committed`).
- Offset yang di-commit = offset pesan **+ 1**.

NestJS (`@nestjs/microservices`, di atas kafkajs)

```
app.connectMicroservice<MicroserviceOptions>({
  transport: Transport.KAFKA,
  options: {
    postfixId: '',
    client: { clientId: 'payment-service', brokers },
    consumer: { groupId: 'payment-service',
        allowAutoTopicCreation: false },
    run: { autoCommit: false, partitionsConsumedConcurrently: 1
        },
    subscribe: { fromBeginning: true },
  },
});
app.enableShutdownHooks();
await app.startAllMicroservices();
```

- Producer: `ClientsModule` dengan `producerOnlyMode: true`, `send: { acks: -1 }`.
- Di kafkajs, `autoCommit` ada di `run`, `fromBeginning` di `subscribe` (beda dengan klien Confluent).
- Commit manual: `context.getConsumer().commitOffsets([{ topic: context.getTopic(), partition: context.getPartition(), offset: (BigInt(context.getMessage().offset) + 1n).toString() }])`.
- `@EventPattern`: exception yang tidak ditangani dianggap retriable (pesan diulang). Batasi dengan retry topic + DLT.
- `postfixId: ''` mencegah Nest menambah `-client`/`-server` ke `clientId` dan `groupId`.

Pola reliability

Masalah	Pola
Event datang dua kali	`INSERT INTO processed_events (event_id) ... ON CONFLICT DO NOTHING` dalam transaksi yang sama dengan efek ke database, lalu commit offset
Gagal sementara	Retry topic dengan header `retry-count`, `next-retry-at`, maksimal N kali
Gagal permanen	DLT dengan header `error-message`, `original-topic`, `original-partition`, `original-offset`; perbaiki penyebab, lalu replay
DB dan Kafka harus berubah bersama	Outbox: baris bisnis + baris outbox dalam satu transaksi DB, publisher `FOR UPDATE SKIP LOCKED`
Kafka ke Kafka atomik	Transaksi Kafka (`sendOffsets`), consumer `read_committed`

Urutan aman di handler: validasi envelope, klaim `eventId`, tulis efek ke database, **COMMIT DB**, baru commit offset.

Envelope dan header

`eventId` (UUID), `eventType` (`order.created.v1`), `version`, `occurredAt` (ISO-8601), `correlationId`, `causationId` (opsional), `data`. Header: `correlation-id`, `event-type`, `traceparent` (`00-<32 hex>-<16 hex>-01`).

SASL/SCRAM (klien Confluent)

```
kafkaJS: { brokers, ssl: true, sasl: { mechanism: 'scram-sha-
        512', username, password } },
'ssl.ca.location': '/path/ca.pem', // kalau CA internal
```

Lab memakai `SASL_PLAINTEXT` di `localhost:9192-9194` (`ssl: false`); produksi memakai `SASL_SSL`.

Perintah CLI (dari folder lab)

```
K=/opt/kafka/bin
docker compose exec kafka-1 $K/kafka-topics.sh --bootstrap-server
        kafka-1:29092 --describe --topic order.created.v1
docker compose exec kafka-1 $K/kafka-consumer-groups.sh --
        bootstrap-server kafka-1:29092 --describe --group
        payment-service
# replay: group harus tidak aktif dulu
docker compose exec kafka-1 $K/kafka-consumer-groups.sh --
        bootstrap-server kafka-1:29092 \
  --group payment-service --topic order.created.v1 --reset-
        offsets --to-earliest --execute
docker compose exec kafka-1 $K/kafka-console-consumer.sh --
        bootstrap-server kafka-1:29092 \
  --topic order.created.v1 --from-beginning --formatter-property
        print.headers=true
```

Angka yang perlu diingat

RF – `min.insync.replicas` = jumlah broker yang boleh mati saat menulis. Consumer aktif ≤ jumlah partisi. Lag = log end offset – committed offset; lag 0 belum tentu sehat. Log `group coordinator is not available` beberapa detik setelah cluster menyala itu normal.